

Quality Classification of Ceramic Tiles using Neural Networks (DNN & CNN)

Jeison Nicolas Diaz Arciniegas
jediazar@unal.edu.co

David Santiago Nagles Barajas
dnagales@unal.edu.co

Miguel Antonio Parrado Pardo
mparradop@unal.edu.co

—
Prof. Eng D. Jorge Ivan Sofrony Esmeral
Universidad Nacional de Colombia - Engineering Faculty
Artificial Intelligence Techniques
Bogota. Colombia

I. INTRODUCTION.

The objective of this project is to develop a classifier system for the quality of ceramic tiles, from three different operation variables in three industrial process stages. These stages are Pressing, Drying and Baking. The data given contains measurements of each stage in a time frame of hours, with an additional register of the batch quality divided in two categories: First and Scrap.

Getting more in depth, the first step taken was to do an Exploratory Data Analysis [EDA], followed by the modeling of two Neural Networks; a Dense Neural Network (DNN) and a Convolutional Neural Network (CNN). These models would be optimized by a given set of HyperParameters, and then evaluated by performance, comparing the best metrics obtained by each network.

II. DATA PRE-PROCESSING.

This section is divided in four parts, as presented here:

- Data reading and cleaning:
- Frequency adjustments of the dataset.
- Time Windows building.
- Variable Normalization.

A. Data reading and cleaning.

The data was given from the file Ceramic_process_QC.xlsx; which has the next sheets:

- prensas
- secadero
- horno
- calidad

After reading the data, the columns containing time labels (fecha, hora, hora_sensor where the columns) are considered irrelevant, and thus deleted.

B. Frequency adjustments to the dataset.

The sheets "horno" and "calidad" had data from each sensor in time frames of 30 minutes, while the rest of the data had its values registered every hour. For this, the next equation was used in every row containing the information in sets of 13 minutes:

$$row_n^{new} = \frac{row_{2n-1}^{past} + row_{2n}^{past}}{2} \quad (1)$$

What this does is finding the average measurements of the data, in half the frequency; which results in a halfling of the data of every variable with a 30 minute window. The number of rows went from 4390 to 2195.

C. Time Windows building

Thinking forwards to the training of the Neural Networks, Time Windows need to be considered for binning the data. A Time Window of value $W = 5$ is chosen, by selecting 5 samples of the quality sheet. This value is also considered an HyperParameter.

D. Variable Normalization.

Min-Max normalization is used to normalize the variables. This method of normalization is chosen to get some stability in the network training, and to achieve a faster learning. It's also chosen because the upper and lower limits of the data is approximately known, but the data distribution is still unknown.

The formula used in Min-Max normalization is as follows:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2)$$

III. EXPLORATORY DATA ANALYSIS. [EDA]

The EDA was performed for each sensor in each variable, in the steps shown below:

- Temporal Series by Variable.
- Histograms and Distributions.
- Correlation Matrix.

A. Temporal Series by Variable.

By plotting the data taken of each sensor for variables "secadero", "prensa" and "horno" against time, it can be seen how each industrial process behaves, and how correlated they are with each other.

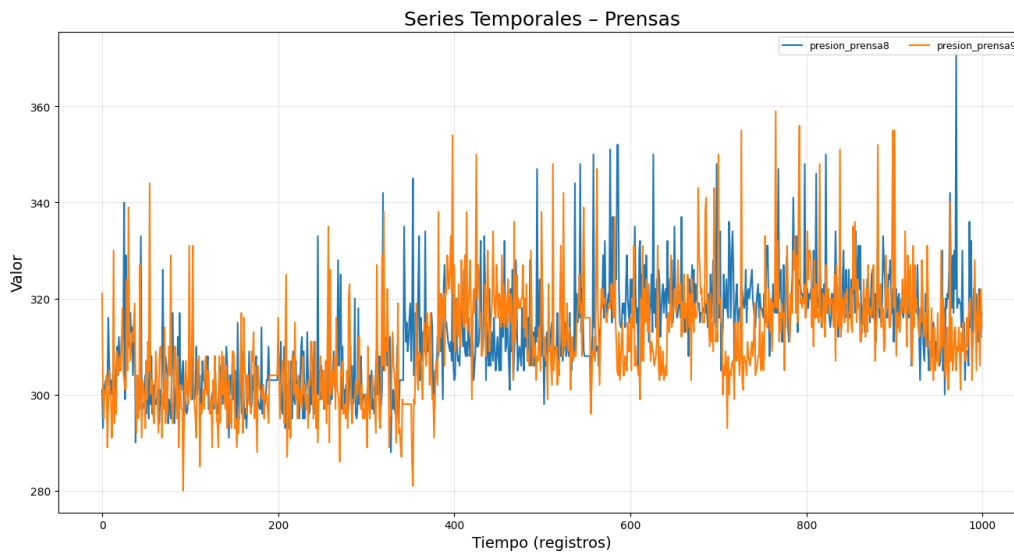


Figure 1: Data for variable "prensa"

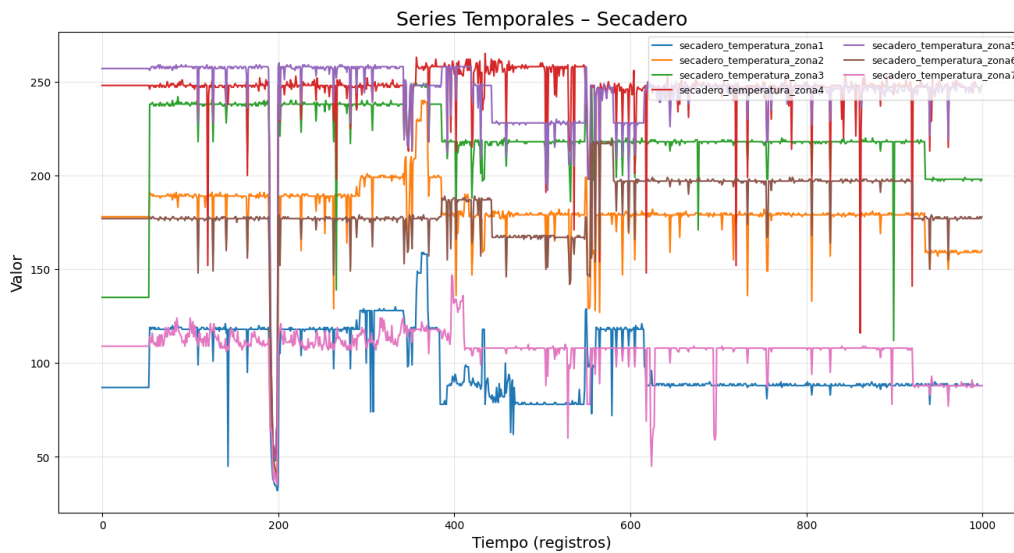


Figure 2: Data for variable "secadero"

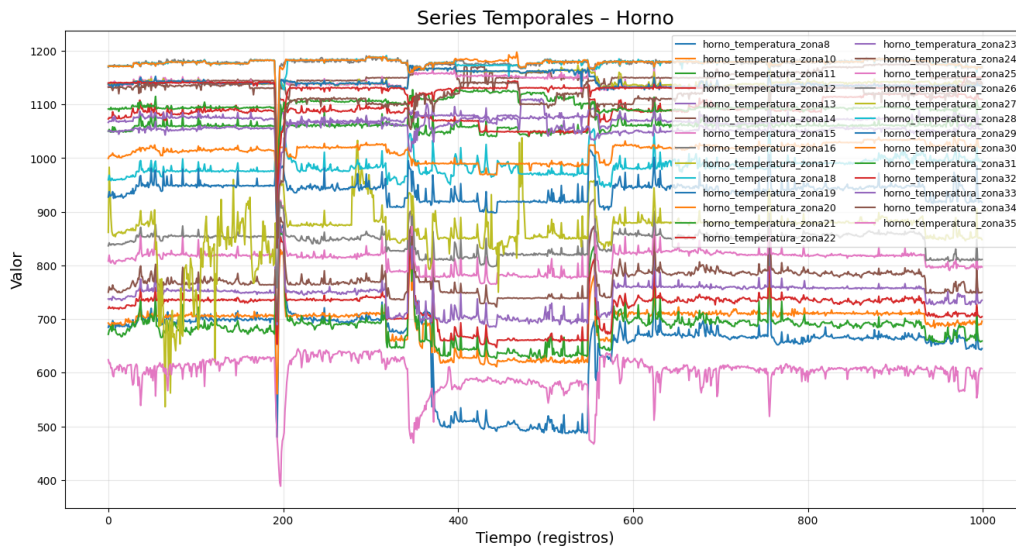


Figure 3: Data for variable "horno"

While the graphs are chaotic, there are some patterns that can be seen. Looking at Figure 1, at around time 400 there is an increase in the pressure given to the ceramic tile, registered by the sensors. Similarly, in Figure 1 there are stable periods for all the data captured by the sensor between time 200 and 400, as well as 600 to 950. And in Figure 3, there is a peak registered in all sensors at around time 200.

The data obtained in this manner is not particularly useful, but a general idea is given on where to search for commonalities in the data.

B. Histograms and Distributions.

For a better comprehension of the processed data, histograms were made to more easily understand what kinds of distributions the data has, and what patterns emerge. The histograms are shown below:

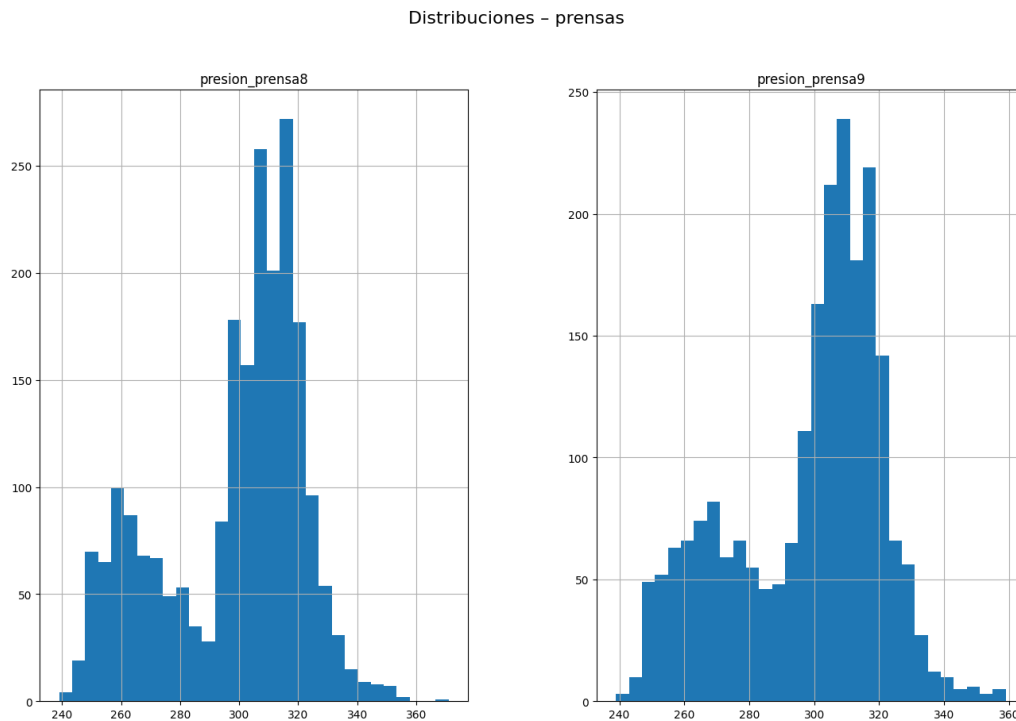


Figure 4: Histogram for variable "prensa"

Distribuciones - secadero

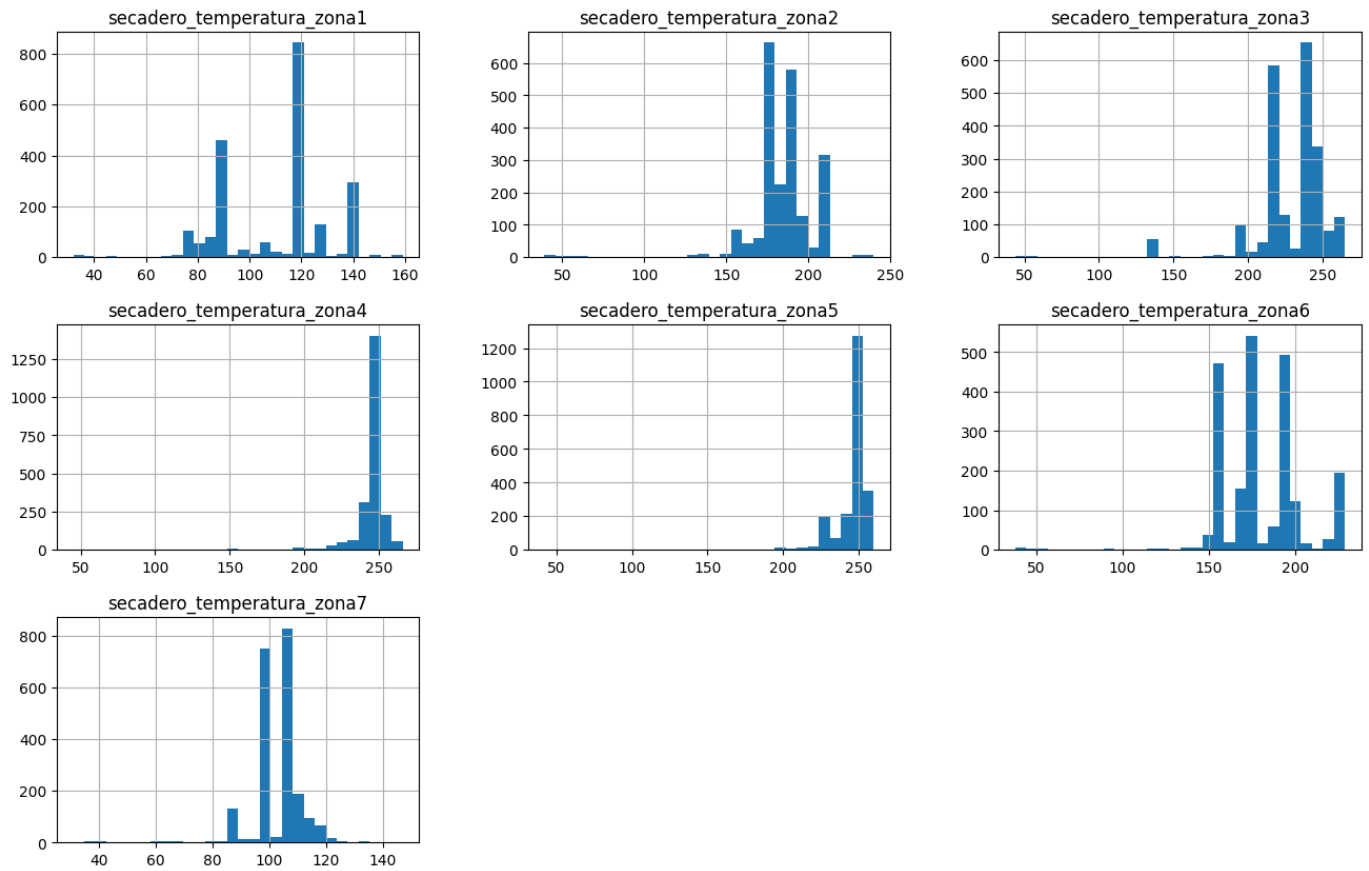


Figure 5: Histogram for variable "secadero"

Distribuciones - horno

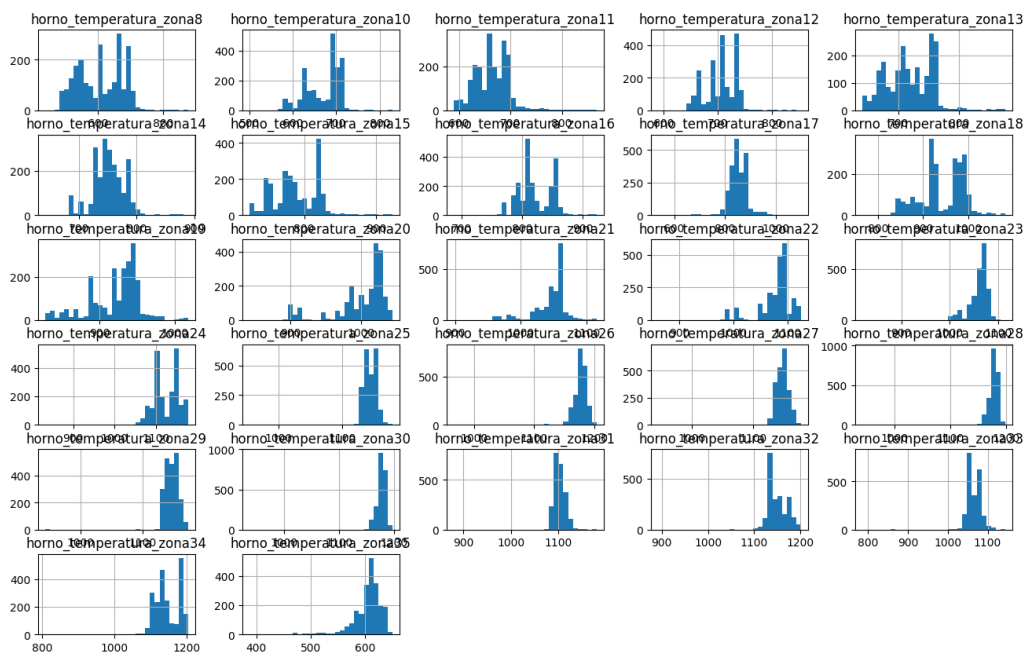


Figure 6: Histogram for variable "horno"

Distribuciones - calidad

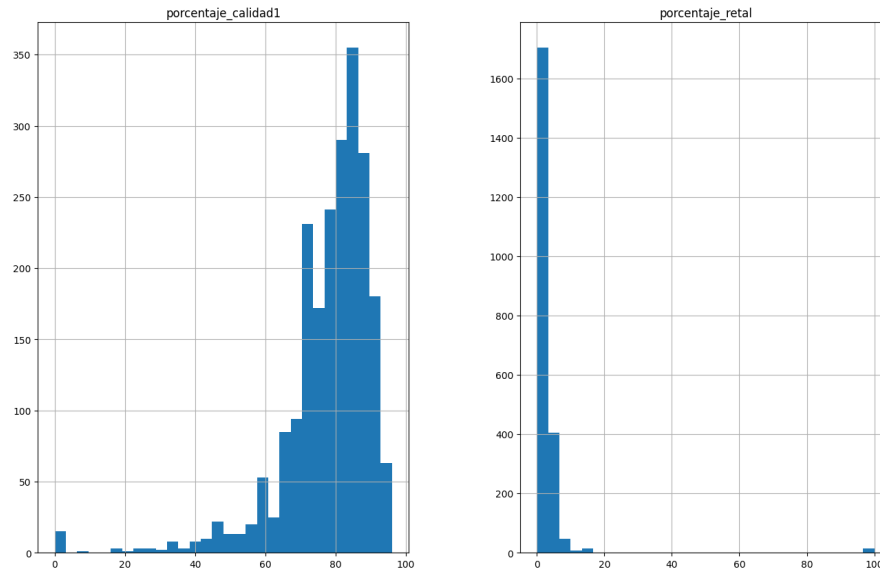


Figure 7: Histogram for variable "calidad"

The histograms in Figure 4 for variable "prensa", show approximately the same distribution for each of its sensors; but it does not show a discernible correlation to any of the other histogram shapes.

The histograms in the Figures 6 and 5 show some similar distributions one to the other, especially the ones that group the data to the right side of the plot. This fact remains important when comparing those distributions to the two histograms shown at Figure 7, where comparing to variable "quality", most specifically to the left side graph, they show a similar shape and form, indicating some form of correlation.

C. Correlation Matrix.

Via Seaborn, a matplotlib based python library, a correlation heatmap matrix was made, to facilitate the recognition of similar variables.

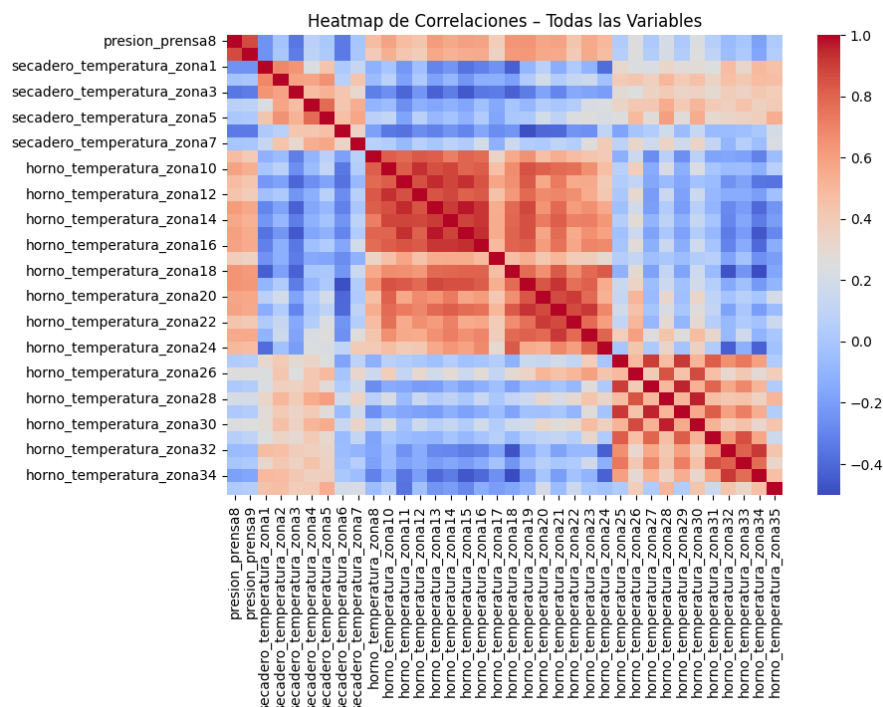


Figure 8: Correlation Heatmap Matrix

As shown in figure 8, there are correlations among the variables in the "secadero" sensors, and a slight correlation with sensors 25 to 35 of the "horno" variable.

The "prensa" sections have strong correlations among themselves, and a slight correlation with zones 8 to 24 of the "horno" variable.

The "horno" variable is correlated among its zones; more specifically, there are groups from zones 8 to 16, 18 to 24, and 25 to 35. These correlations vary between strong and weak.

Given that the correlation of each variable its the strongest with itself, another plot is made, showing the correlation of all data taken by the sensors, with the variable "calidad". which is the criterion that decides what batch tiles pass for first quality classification.

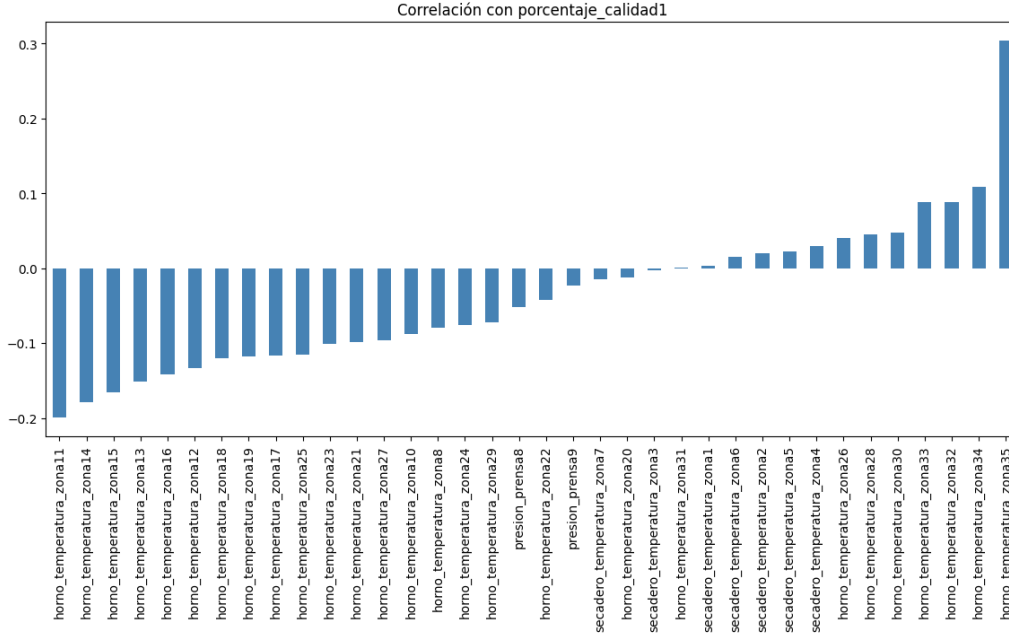


Figure 9: Correlation of each sensor variable to the quality of the tile.

Grom all the information above, is decided to:

- Train the models with all the data, even if it shows low correlation with variable "calidad".
- Use a 50/50 split to decide which ceramic tiles classify for first quality, and which ones classify as scrap.

IV. NEURAL NETWORK MODELING

Before modeling, the dataset was created with a batch size of 32 and a time window of $W = 5$ hours as discussed previously. Two arrays containing features and labels are created; these are then packaged into a tensor using PyTorch. 70% of this dataset is used for training, and the remainder is used for testing.

From this dataset, two models where trained, one being a Dense Neural Network, and the other being a Deep Neural Network.

A. Dense Neural Network [DNN]

This model was programmed with PyTorch, using ReLU as the activation function, making use of the AdaM optimizer, with Cross Entropy Loss criteria, and mini-batch training. 20 epochs were used for its training.

The following HyperParameters were tested to find the most optimal ones.

- Number of hidden layers: [1, 2, 3, 4, 5, 6]
- Neurons by Layer: [20, 50, 100, 150]
- Learning Rates: [0.01, 0.001, 0.0001]

The best results were obtained with an Learning Rate of 0.01, 1 hidden layer, and 20 neurons per layer; with the following scores:

Model Score Results	
Accuracy	0.9470
Precision	0.9470
Recall	1.0000
F1	0.9728

Table I: DNN Scores

The confusion Matrix and the loss per epoch are as follows:

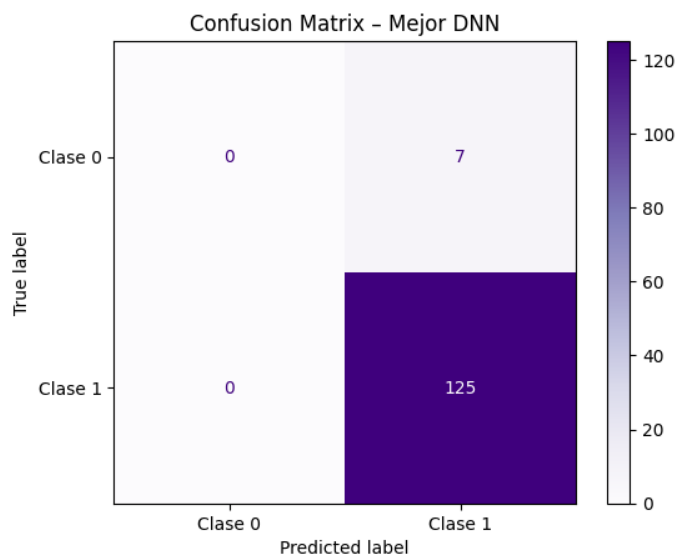


Figure 10: Confusion Matrix for the DNN

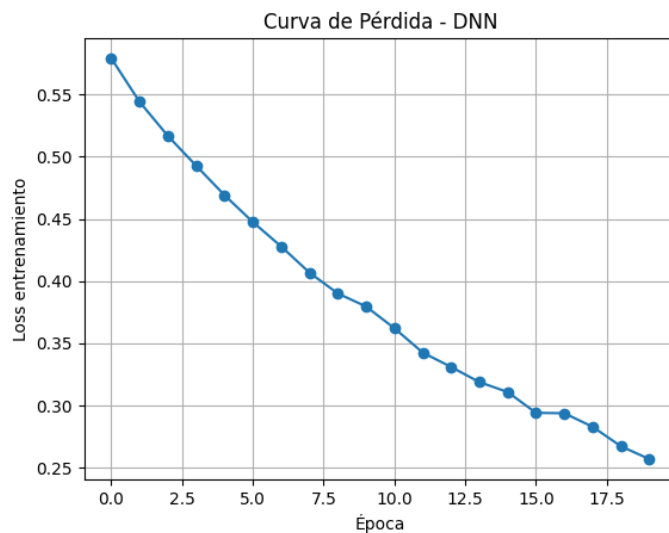


Figure 11: Training Loss for the DNN

The high scores obtained by the model can be explained, given that the dataset provided was somewhat imbalanced, and that the classification split was a 50/50.

For an easier visualization of the HyperParameters optimization for the DNN, the following plots are provided:

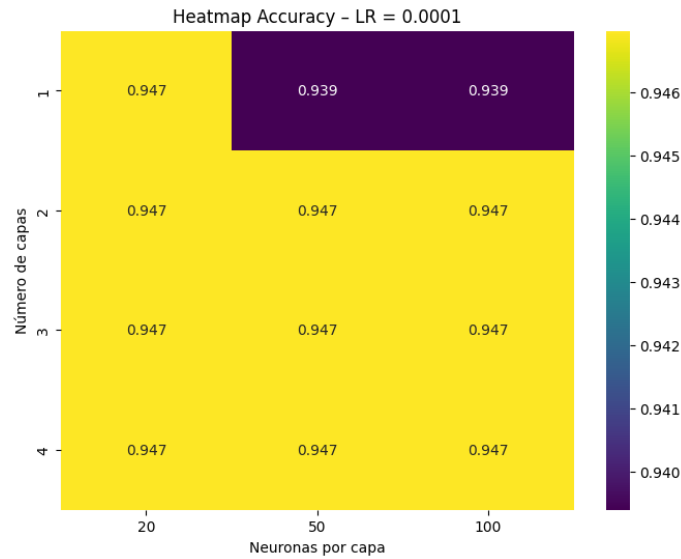


Figure 12: Accuracy of Neurons against Layers for the DNN

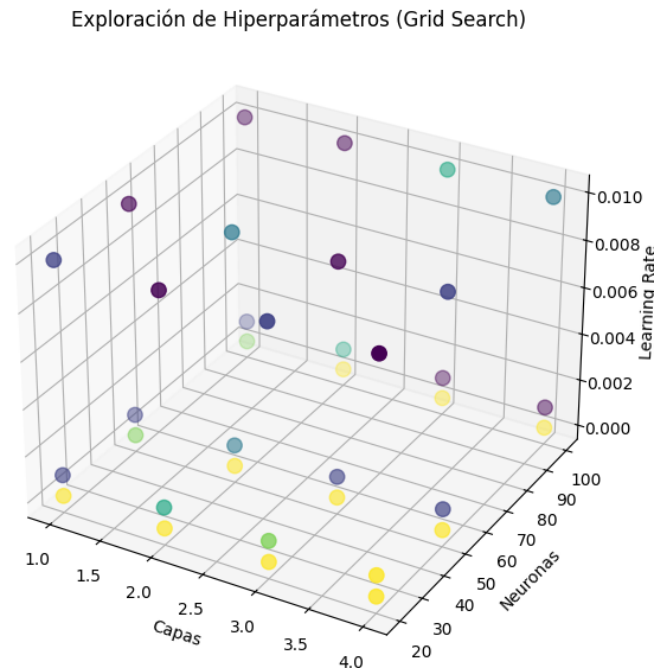


Figure 13: 3D Grid Search for Accuracy of the Layers, Neurons, and Learning Rate HyperParameters.

Figure 12 and 13 share the same scale, where colors that go to the yellows contain a better accuracy, and colors that go to the purple show a lower accuracy. It can be seen that, while the color contrast it's very sharp, the accuracy only differentiates a 0.009 from highest obtained, to lowest obtained.

B. Convolutional Neural Network [CNN]

the model was designed flexibly. By treating the variables as 5x36 matrices, spatial dependencies between the sensors can be captured.

For the experiments, grid search was performed to identify promising configurations.

The following HyperParameters were tested:

- Learning rates: 0.01, 0.001]
- Number of Kernels: [8, 16]
- Kernel size: [2x2, 3x3]

- Pooling type: [Maximum (max), Average (avg)]
- Number of Fully Connected Neurons: [16, 32, 64] Number of Fully Connected Neuron Layers: [1, 2]

The best results were obtained with a Learning Rate of 0.01, 8 2x2 size Kernels, Maximum pooling, 16 Fully Connected Neurons, and a single Fully Connected Layer. The next scores where achieved by the model:

Model Score Results	
Accuracy	0.9545
Precision	0.9545
Recall	1.0000
F1	0.9542

Table II: DNN Scores

Confusion Matrix of this model is as follows;

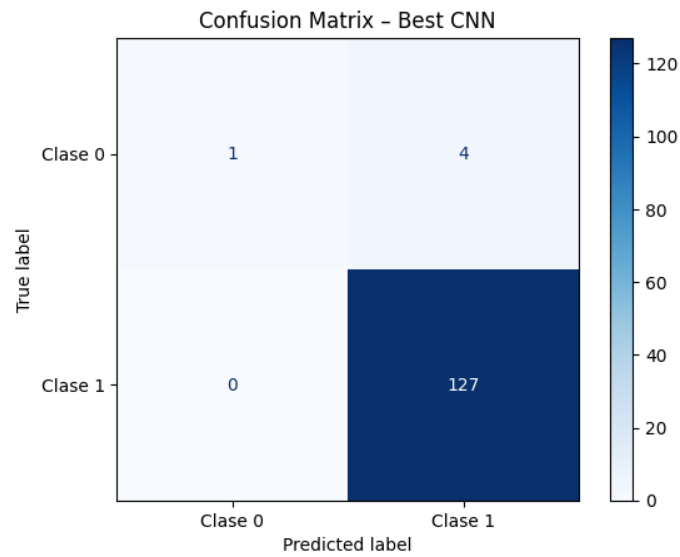


Figure 14: Confusion Matrix for the CNN

And grid search optimization for HyperParameters:

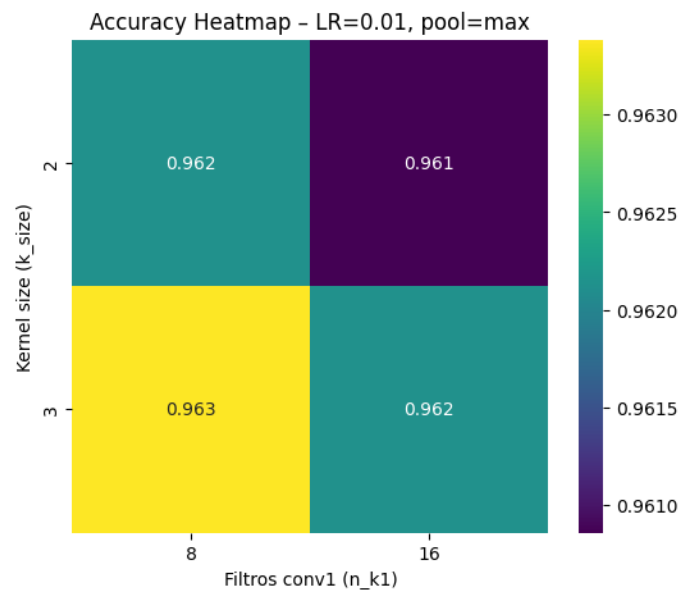


Figure 15: Accuracy of Neurons for max type pooling.

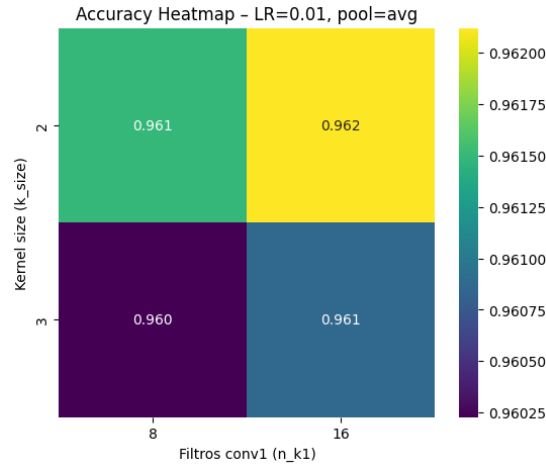


Figure 16: Accuracy of Neurons for avg type pooling.

C. Cross-Validation

Sequential cross-validation was implemented, with 3 folds; and using the same data loaders for each model. The results are shown in the figure below:

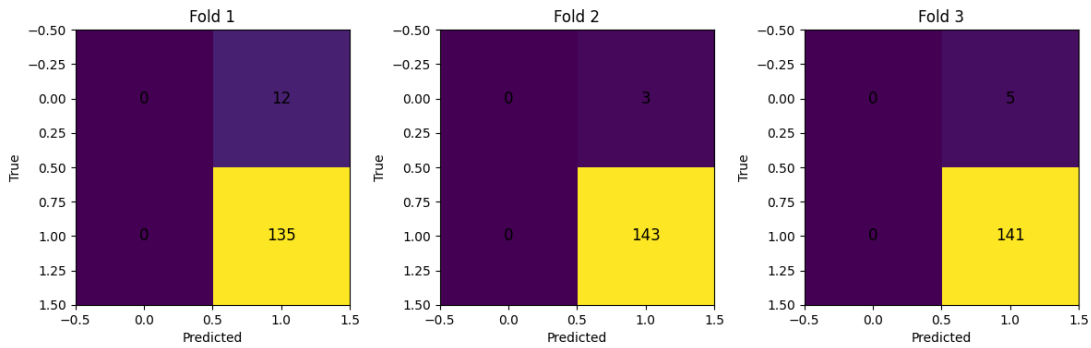


Figure 17: 3-Fold Cross Validation of the DNN and CNN

V. CONCLUSIONS

We can conclude, as we had already observed in previous practices, that data quality is a decisive factor in the models performance. We confirmed that the decisions made by the model depend largely on how the data are distributed and represented. In our case, since the dataset is not balanced, we observed that the model tends to classify almost all samples as first-class. This is expected, given that it was trained mostly with data from that category.

We also found that our approximation assuming that tiles with more than 50% of their surface in good condition belong to the first class introduces an additional bias. By not properly separating second-class and leftover (retal) categories, the model loses relevant information and replicates these biases during prediction.

In conclusion, it is reasonable to expect that by improving and balancing the dataset, we can achieve better performance and more accurate models.

Regarding the considerations applied during training, we implemented the cross-entropy loss function, which proved to be appropriate for our multiclass classification problem. This choice allowed us to properly evaluate the discrepancy between the true class labels and the models predictions, unlike the loss functions typically used in regression tasks.

We also employed the Adam optimizer, which contributed to improving the training process thanks to its ability to dynamically adjust the learning rate. This led to a more stable and efficient convergence, which was reflected in the models overall performance.

We can observe that the CNN did not fall behind the DNN in terms of performance, which makes sense given the nature of our data. With the CNN, our goal was to capture spatial patterns, and in this case we can see that there is indeed a degree of dependency between consecutive data points, since we are dealing with time-series information. However, it is important to note that the difference in performance between both models is not particularly significant or concerning.